

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

```
template.cpp119 lines
#include<bits/stdc++.h>
using namespace std;

// Extra functionality :
// *st.find_by_order(index) = value at index
// st.order_of_key(x) = no. of elements strictly less than x

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define ordered_set tree<int, null_type,less<int>, \
rb_tree_tag,tree_order_statistics_node_update>

#define int long long
#define ll long long
#define lld long double
#define vi vector<int>
#define pll pair<ll, ll>
#define vll vector<ll>
#define vvl vector<vll>
#define vvll vector<vvll>
#define ar array
#define F first
#define S second

#define all(v) v.begin(),v.end()
#define range(v, i, j) v.begin()+i, v.begin()+j+1
#define rep(i, a, b) for(long long i = (a); i < (b); ++(i))
#define L(i, a, b) for(long long i = (a); i <= (b); ++(i))
#define R(i, a, b) for(long long i = (a); i >= (b); --(i))
#define sz(x) (ll)(x.size())
#define extract(m, x) \
{ auto it = (m).find(x); if (it != (m).end()) (m).erase(it); }
#define gp " "
#define nl "\n"

#define isSet(x, i) ((x>>i)&1)
#define setbit(x, i) (x | (1LL<<i))
```

```
1#define resetbit(x, i) (x & (~(1LL << i)))
2#define toggleBit(x, i) ((x) ^ (1LL << (i)))
3#define clz(x) __builtin_clzll(x)
4#define ctz(x) __builtin_ctzll(x)
5#define csb(x) __builtin_popcountll(x)
6#define msb(x) ((x) ? (63 - __builtin_clzll((ll)(x))) : -1)
7#define lsb(x) ((x) ? (__builtin_ctzll((ll)(x))) : -1)
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
254
```

```
    echo "Passed test: " $i
done
echo "WA on test:"
cat tcase
echo "Your answer:"
cat myans
echo "Correct answer:"
cat corans
```

```
gen.cpp
4 lines
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
    count());
inline ll rnd(ll l, ll r) {
    return uniform_int_distribution<ll>(l, r)(rng);
}
```

```
brute.cpp
1 lines
// Write brute force for stress testing
```

```
d.bat
8 lines
@echo off /// set SRC=code.cpp /// set EXE=a
g++ -std=c++17 -DLOCAL -Wshadow -Wall -g -D_GLIBCXX_DEBUG -fmax
    -errors=2 %SRC% -o %EXE%
if errorlevel 1 (
    echo Compilation failed! /// exit /b
)
echo. /// echo Input: /// type %1 /// echo.
echo. /// echo Output: /// %EXE% < %1 /// echo.
/// alt : gnu++14 or ++11 or ++1z
```

```
mint.h
40 lines
const int mod=1000000007;
struct mint{
    ll x; mint() { x=0; }
    mint(ll _x) {x = _x%mod; if(x<0)x+=mod; }
```

```

    mint operator+=(mint b){ x= (x + b.x)%mod; return *this; }
    mint operator-=(mint b){ x= (x-b.x +mod)%mod; return *this; }
    mint operator*=(mint b){ x= (x * b.x)%mod; return *this; }
    mint operator/=(mint b){ return *this *= b.inv(); }
```

```

    friend mint operator+(mint a,mint b){ return a+=b; }
    friend mint operator-(mint a,mint b){ return a-=b; }
    friend mint operator*(mint a,mint b){ return a*=b; }
    friend mint operator/(mint a,mint b){ return a/=b; }
    mint operator-() { return mint()-*this; }
    mint inv() { return power(mod-2); }
```

```

    mint power(ll n) {
        mint a=*this, res=1;
        while(n) { if(n&1)res*=a; a*=a; n>>=1; }
        return res; }
```

```

    explicit operator bool() const { return x != 0; }
    friend ostream& operator<<(ostream& os, const mint& m) \
        { return os << m.x; }
```

```
};
mint power(mint a,ll n){ return a.power(n); }
```

```
const int N=2000005; mint fact[N],inv_fact[N];
void prep_factorials(){
    fact[0] = 1;
    for(int i=1;i<N;++i) fact[i] = fact[i-1] * i;
    inv_fact[N-1] = fact[N-1].inv();
    for(int i=N-2;i>=0;--i) inv_fact[i]=inv_fact[i+1]*(i+1);
```

```

}

mint nCr(int n,int r){
    if(r<0 || r>n) return mint();
    return fact[n] * inv_fact[r] * inv_fact[n-r];
}
```

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g.
 $a_n = (d_1n + d_2)r^n$.

2.3 Trigonometry

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

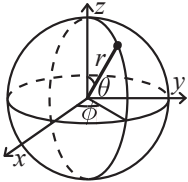
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° ,
 $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1 - x^2}} \qquad \frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1 - x^2}}$$

$$\frac{d}{dx} \tan x = 1 + \tan^2 x \qquad \frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$$

$$\int \tan ax = -\frac{\ln |\cos ax|}{a} \qquad \int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \text{erf}(x) \qquad \int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax - 1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a+c^{a+1}+\cdots+c^b=\frac{c^{b+1}-c^a}{c-1}, c\neq 1$$

$$1+2+3+\cdots+n=\frac{n(n+1)}{2}$$

$$1^2+2^2+3^2+\cdots+n^2=\frac{n(2n+1)(n+1)}{6}$$

$$1^3+2^3+3^3+\cdots+n^3=\frac{n^2(n+1)^2}{4}$$

$$1^4+2^4+3^4+\cdots+n^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x=1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots, (-\infty < x < \infty)$$

$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x}=1+\frac{x}{2}-\frac{x^2}{8}+\frac{2x^3}{32}-\frac{5x^4}{128}+\ldots, (-1 \leq x \leq 1)$$

$$\sin x=x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots, (-\infty < x < \infty)$$

$$\cos x=1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots, (-\infty < x < \infty)$$

Data structures (3)

OrderedMultiset.h

Description: ...

Time: ... 93fcc8, 22 lines

```
template <typename T>
struct ordered_multiset {
    using Tpair = pair<T, int>;
    tree<Tpair, null_type, less<Tpair>, rb_tree_tag \
        tree_order_statistics_node_update> t;
    int _idx = 0;
    void insert(const T &x) { t.insert({x, _idx++}); }
    void erase(const T &x) {
        auto it = t.lower_bound({x, 0});
        if (it != t.end() && it->first == x) t.erase(it);
    }
    int order_of_key(const T &x) const {
        return t.order_of_key({x, 0});
    }
    T find_by_order(int k) const {
        auto it = t.find_by_order(k);
        if (it == t.end()) return 1000000000;
        return it->first;
    }
    int size() const { return t.size(); }
    bool empty() const { return t.empty(); }
};
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: O(log N) ea56ff, 36 lines

```
using T = ll;
struct Seg {
    int n, m=1;
    vector<T> t;
    const T unit = -inf; // identity element

    Seg(int _n): n(_n) {
        while(m<n) m<<=1;
        t.assign(2*m, unit);
    }
    T f(T a, T b) { return max(a, b); } // operation

    void upd(int p, T v) {
        for(t[p+=m]=v; p>1; p>>=1)
            t[p>>=1] = f(t[p&~1], t[p|1]); // parent, lef, right
    }
    T qry(int l, int r) { // [l, r)
        T L=unit, R=unit;
        for(l+=m, r+=m; l<r; l>>=1, r>>=1) {
            if(l&1) L = f(L, t[l++]);
            if(r&1) R = f(t[--r], R);
        }
        return f(L,R);
    }
    // first position with value >= v
    int first(T v) {
        if(t[1] < v) return -1;
        int p = 1;
        while(p < m) {
            if(t[2*p] >= v) p = 2*p;
            else p = 2*p+1;
        }
        int res = p - m;
        return (res < n) ? res : -1;
    }
};
```

LazySegmentTree.h

Description: can use set and add together. ranges are all [], EVEN IN CONSTRUCTOR TOO Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.

Usage: Lseg seg(a, 1, n+1); / seg(1, n+1) / seg(0, n)

Time: O(log N).

"../various/BumpAllocator.h" 4ad771, 65 lines

```
using T = ll;
struct Node {
    T mn, mx, sm; // ignore variables you dont need
    Node() : mn(-inf), mx(-inf), sm(0) {}
    Node(T v) : mn(v), mx(v), sm(v) {}
    Node(T _mn, T _mx, T _sm) : mn(_mn), mx(_mx), sm(_sm) {}
};
struct Lseg {
    // s:set tag (inf=none), a:add tag, v:stored node values
    Lseg *lc=0, *rc=0; int l, r; T s=inf, a=0; Node v;
    // Associative merge function
    Node f(Node i, Node j) {
        return {min(i.mn, j.mn), max(i.mx, j.mx), i.sm + j.sm};
    }
    // Huge interval of neutrals
    Lseg(int _l, int _r): l(_l), r(_r) {}
    Lseg(vll& vc, int _l, int _r): l(_l), r(_r) {
        if (l+1 < r) {
            int m = l+(r-1)/2;
            lc = new Lseg(vc, l, m); rc = new Lseg(vc, m, r);
```

```
            v = f(lc->v, rc->v);
            } else if(l < (int)vc.size()) v = Node(vc[l]);
        }
        // apply set
        void ps(T x) { s=v.mn=v.mx=x, a=0, v.sm=x*(r-1); }
        // apply add
        void pa(T x) {
            (s!=inf ? s : a)+=x, v.mn+=x, v.mx+=x, v.sm+=x*(r-1);
        }
        // push
        void ph() {
            int m = l+(r-1)/2;
            if(!lc) lc=new Lseg(l, m), rc=new Lseg(m, r);
            if(s!=inf) lc->ps(s), rc->ps(s), s=inf;
            if(a) lc->pa(a), rc->pa(a), a=0;
        }
        // Range Set: Update all elements in [x, y) to val
        void set(int x, int y, T val) {
            if(y<=l || r<=x) return;
            if(x<=l && r<=y) ps(val);
            else ph(), lc->set(x, y, val), rc->set(x, y, val), \
                v=f(lc->v, rc->v);
        }
        // Range Add: Add val to all elements in [x, y)
        void add(int x, int y, T val) {
            if(y<=l || r<=x) return;
            if(x<=l && r<=y) ps(val);
            else ph(), lc->add(x, y, val), rc->add(x, y, val), \
                v=f(lc->v, rc->v);
        }
        // Returns Node containing mn, mx, sm of [x, y)
        Node qry(int x, int y) {
            if(y<=l || r<=x) return Node();
            if(x<=l && r<=y) return v;
            ph(); return f(lc->qry(x, y), rc->qry(x, y));
        }
        // First index in [x, y) with value >= val
        int fst(T val, int x, int y) {
            if(y<=l || r<=x || v.mx<val) return -1;
            if(l+1 == r) return l;
            ph(); int res = lc->fst(val, x, y);
            return (res != -1) ? res : rc->fst(val, x, y);
        }
        ~Lseg() { delete lc; delete rc; }
};
```

PersistentSeg.h

Description: ...

Usage: vector<Node *> pref(n+1);
pref[0] = build(0, unqvals); // [0, unqvals)
trees[k] = change(trees[k-1], 0, unqvals, pos, x);
cout << query(trees[k], 0, unqvals, 1, r+1) << nl;
Time: ... a4e183, 34 lines

```
struct Node {
    Node *left = nullptr, *right = nullptr; ll sum = 0;
};
// Builds a segment tree on [l,r). use same [l,r) in qry, upd
// Returns a pointer to the constructed tree.
Node *build(int l, int r) {
    Node *p = new Node();
    if (r - l == 1) return p;
    int m = (l + r) / 2;
    p->left = build(l, m);
    p->right = build(m, r);
    return p;
}
// Changes value in position k to x in tree p.
// Returns a pointer to the new tree.
```

```
Node *change(Node *p, int l, int r, int k, int x) {
    Node *n = new Node(*p);
    if (r - l == 1) { n->sum = x; }
    else {
        int m = (l + r) / 2;
        if (k < m) { n->left = change(p->left, l, m, k, x); }
        else { n->right = change(p->right, m, r, k, x); }
        n->sum = n->left->sum + n->right->sum;
    }
    return n;
}
// Returns the sum of values in range [ql, qr].
ll query(Node *p, int l, int r, int ql, int qr) {
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return p->sum;
    int m = (l + r) / 2;
    return query(p->left, l, m, ql, qr) +
           query(p->right, m, r, ql, qr);
}
```

SegTree2D.h

Description: ...
Usage: one based grid, and all inclusive
Time: log square n

3e4e8d, 30 lines

```
const int TREE_SIZE = 1 << 10;
int trees[TREE_SIZE * 2][TREE_SIZE * 2];
// Adds d to the position x in the segment tree 'tree'.
void add(int *tree, int x, int d) {
    x += TREE_SIZE;
    while (x > 0) { tree[x] += d; x /= 2; }
}
// Adds d to the position (i, j).
void add(int y, int x, int d) {
    y += TREE_SIZE;
    while (y > 0) { add(trees[y], x, d); y /= 2; }
}
// Returns the sum of value in range [x1, x2] in 'tree'.
int query(int *tree, int x1, int x2) {
    x1 += TREE_SIZE; x2 += TREE_SIZE; int sum = 0;
    while (x1 <= x2) {
        if (x1 % 2 == 1) sum += tree[x1++];
        if (x2 % 2 == 0) sum += tree[x2--];
        x1 /= 2; x2 /= 2;
    } return sum;
}
// Returns the sum of value in the rectangle [y1, y2]x[x1, x2]
int query(int y1, int y2, int x1, int x2) {
    y1 += TREE_SIZE; y2 += TREE_SIZE; int sum = 0;
    while (y1 <= y2) {
        if (y1 % 2 == 1) sum += query(trees[y1++], x1, x2);
        if (y2 % 2 == 0) sum += query(trees[y2--], x1, x2);
        y1 /= 2; y2 /= 2;
    } return sum;
}
}
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip : st, time() and rollback().
Usage: int t = uf.time(); ...; uf.rollback(t);
Time: $\mathcal{O}(\log(N))$

05a554, 26 lines

```
// c = current no. of components
// time() returns current checkpoint
// rollback(x) rolls back to checkpoint x
struct DSU {
    vi e; vector<pii> st; int c;
    DSU(int n) : e(n, -1), c(n) {}
    int size(int x) { return -e[find(x)]; }
```

```
int find(int x) { return e[x] < 0 ? x : find(e[x]); }
int time() { return sz(st); }
void rollback(int t) {
    c += (time() - t) / 2;
    for (int i = time(); i --> t; )
        e[st[i].first] = st[i].second;
    st.resize(t);
}
bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    --c;
    return true;
}
};
```

SubMatrix.h

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // top left 4 elements
Time: $\mathcal{O}(N^2 + Q)$

c59ada, 13 lines

```
template<class T>
struct SubMatrix {
    vector<vector<T>>> p;
    SubMatrix(vector<vector<T>>& v) {
        int R = sz(v), C = sz(v[0]);
        p.assign(R+1, vector<T>(C+1));
        rep(r,0,R) rep(c,0,C)
            p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
    }
    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};
```

Matrix.h

Description: Basic operations on square matrices.
Usage: Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;

f71c23, 31 lines

```
template<class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i,0,N) rep(j,0,N)
            rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
    array<T, N> operator*(const array<T, N>& vec) const {
        array<T, N> ret{};
        rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
        return ret;
    }
    static M identity() {
        M I;
        rep(i,0,N) I.d[i][i] = 1;
        return I;
    }
    M operator^(ll p) const {
        assert(p >= 0);
```

```
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
        if (p&1) a = a*b;
        b = b*b;
        p >>= 1;
    }
    return a;
}
};
```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

Sec1c7, 30 lines

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

Treap.h

Description: ...
Time: $\mathcal{O}(\log N)$

83ee02, 121 lines

```
// Indexing is 0-based
struct Treap {
    struct Node {
        int val, sum, size, prior, lazy;
        Node *l = nullptr, *r = nullptr;
        Node(int v):val(v),sum(v),size(1), prior(rand()), lazy(0){}
    };
    Node* root = nullptr;

    int _sz(Node* t) { return t ? t->size : 0; }
    int sum(Node* t) { return t ? t->sum : 0; }

    void push(Node* t) {
        if (!t || t->lazy == 0) return;
        t->val += t->lazy;
        t->sum += t->lazy * _sz(t);
```

```

    if (t->l) t->l->lazy += t->lazy;
    if (t->r) t->r->lazy += t->lazy;
    t->lazy = 0;
}

void upd(Node* t) {
    if (!t) return;
    push(t->l); push(t->r);
    t->size = 1 + _sz(t->l) + _sz(t->r);
    t->sum = t->val + sum(t->l) + sum(t->r);
}

void split(Node* t, int k, Node*& l, Node*& r) {
    // l holds first k indices [0...k-1]
    // r holds rem. indices [k...n-1]

    if (!t) { l = r = nullptr; return; }
    push(t);
    if (_sz(t->l) < k) {
        split(t->r, k - _sz(t->l) - 1, t->r, r);
        l = t;
    } else {
        split(t->l, k, l, t->l);
        r = t;
    }
    upd(t);
}

Node* merge(Node* l, Node* r) {
    push(l); push(r);
    if (!l || !r) return l ? l : r;
    if (l->prior > r->prior) {
        l->r = merge(l->r, r); upd(l); return l;
    } else {
        r->l = merge(l, r->l); upd(r); return r;
    }
}

// inserts val at index pos [0 based], shifting the rest right
void insert(int pos, int val) {
    Node *t1, *t2;
    split(root, pos, t1, t2);
    root = merge(merge(t1, new Node(val)), t2);
}

// Range add [l, r]
void range_add(int l, int r, int v) {
    Node *t1, *t2, *t3; // [0...l-1], [l...r], [r+1...n-1]
    split(root, r+1, t2, t3);
    split(t2, l, t1, t2);
    if (t2) t2->lazy += v;
    root = merge(merge(t1, t2), t3);
}

// Range sum [l, r]
int range_sum(int l, int r) {
    Node *t1, *t2, *t3;
    split(root, r+1, t2, t3);
    split(t2, l, t1, t2);
    int ans = sum(t2);
    root = merge(merge(t1, t2), t3);
    return ans;
}

void erase(int pos) {
    // [0...pos-1], [pos...pos], [pos+1...n-1]
    Node *t1, *t2, *t3;
    split(root, pos + 1, t2, t3);
    split(t2, pos, t1, t2);
}

delete t2;
root = merge(t1, t3);
}

// add functions as needed :

void f(ll a, ll b) {
    if(a >= b) return;

    // both cases can be generalized to ...|block|...|block|...
    // any of the ... can be null, no problem

    Node *t1, *t2, *t3, *t4, *t5;
    int blocksize = min(n-b, b-a);
    split(root, b+blocksize, t4, t5);
    split(t4, b, t3, t4);
    split(t3, a+blocksize, t2, t3);
    split(t2, a, t1, t2);
    Node *res = merge(t1, t4);
    res = merge(res, t3);
    res = merge(res, t2);
    res = merge(res, t5);
    root = res;
}

void print() {
    print(root); cout << nl;
}

void print(Node* root) { // simply in-order
    if(!root) return;
    print(root->l);
    cout << root->val << gp;
    print(root->r);
}
};

FenwickTree.h
Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are O(log N).
e62fac, 22 lines

struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos]
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >= 1) {
            if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
                pos += pw, sum -= s[pos-1];
        }
        return pos;
    }
};

FenwickTree2d.h
Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
```

```

    delete t2;
    root = merge(t1, t3);
}

// add functions as needed :

void f(ll a, ll b) {
    if(a >= b) return;

    // both cases can be generalized to ...|block|...|block|...
    // any of the ... can be null, no problem

    Node *t1, *t2, *t3, *t4, *t5;
    int blocksize = min(n-b, b-a);
    split(root, b+blocksize, t4, t5);
    split(t4, b, t3, t4);
    split(t3, a+blocksize, t2, t3);
    split(t2, a, t1, t2);
    Node *res = merge(t1, t4);
    res = merge(res, t3);
    res = merge(res, t2);
    res = merge(res, t5);
    root = res;
}

void print() {
    print(root); cout << nl;
}

void print(Node* root) { // simply in-order
    if(!root) return;
    print(root->l);
    cout << root->val << gp;
    print(root->r);
}
};

FenwickTree.h
Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are O(log N).
e62fac, 22 lines

struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos]
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >= 1) {
            if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
                pos += pw, sum -= s[pos-1];
        }
        return pos;
    }
};

FenwickTree2d.h
Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

Time: $O(\log^2 N)$. (Use persistent segment trees for $O(\log N)$.)

```
"FenwickTree.h"
157f07, 22 lines

struct FT2 {
    vector<vi> ys; vector<FT> ft;
    FT2(int limx) : ys(limx) {}
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
    }
    void init() {
        for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x-1].query(ind(x-1, y));
        return sum;
    }
};

RMQ.h
Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive);
Time: O(|V| log |V| + Q)
510cf32, 16 lines

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, 0, sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

MoQueries.h
Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).
Time: O(N*sqrt(Q))
a12cf4, 49 lines

void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.

Usage: RMQ rmq(values);

rmq.query(inclusive, exclusive);

Time: $O(|V| \log |V| + Q)$

```

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, 0, sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

MoQueries.h
Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).
Time: O(N*sqrt(Q))
a12cf4, 49 lines

void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).

Time: $O(N\sqrt{Q})$

```

void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
```

```
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
}
return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) rep(end,0,2) {
        int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                    else { add(c, end); in[c] = 1; } a = c; }
        while (!(L[b] <= L[a] && R[a] <= R[b]))
            I[i++] = b, b = par[b];
        while (a != b) step(par[a]);
        while (i--) step(I[i]);
        if (end) res[qi] = calc();
    }
    return res;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h	c9b7b0, 17 lines
--------------	------------------

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.

Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0

Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h"	b00bfe, 23 lines
----------------	------------------

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
```

```
vector<double> ret;
Poly der = p;
der.diff();
auto dr = polyRoots(der, xmin, xmax);
dr.push_back(xmin-1);
dr.push_back(xmax+1);
sort(all(dr));
rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
        rep(it,0,60) { // while (h - l > 1e-8)
            double m = (l + h) / 2, f = p(m);
            if ((f <= 0) ^ sign) l = m;
            else h = m;
        }
        ret.push_back((l + h) / 2);
    }
}
return ret;
}
```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.

Time: $\mathcal{O}(n^2)$

	08bf48, 13 lines
--	------------------

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.

Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

Time: $\mathcal{O}(N^2)$

"../number-theory/ModPow.h"	96548b, 20 lines
-----------------------------	------------------

```
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
```

```
    return C;
}
```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.

Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number

Time: $\mathcal{O}(n^2 \log k)$

	f4e444, 26 lines
--	------------------

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

4.2 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

	bd5cec, 15 lines
--	------------------

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.

Time: $\mathcal{O}(N^3)$

	3313dc, 18 lines
--	------------------

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
```

```
ll t = a[i][i] / a[j][i];
if (t) rep(k,i,n)
    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
swap(a[i], a[j]);
ans *= -1;
}
ans = ans * a[i][i] % mod;
if (!ans) return 0;
}
return (ans + mod) % mod;
}
```

SolveLinear.h
Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

```
typedef vector<double> vd;
const double eps = 1e-12;
```

```
int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vi col(m); iota(all(col), 0);

    rep(i,0,n) {
        double v, bv = 0;
        rep(r,i,n) rep(c,i,m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) swap(A[j][i], A[j][bc]);
        bv = 1/A[i][i];
        rep(j,i+1,n) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            rep(k,i+1,m) A[j][k] -= fac*A[i][k];
        }
        rank++;
    }

    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        rep(j,0,i) b[j] -= A[j][i] * b[i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

```
x.assign(m, 0);
for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
}
return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h
Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```
"SolveLinear.h"
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:; }

```

SolveLinearBinary.h
Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

```
typedef bitset<1000> bs;
```

```
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        for (br=i; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if(b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
            A[j] ^= A[i];
        }
        rank++;
    }

    x = bs();
    for (int i = rank; i--;) {
        if (!b[i]) continue;
        x[col[i]] = 1;
        rep(j,0,i) b[j] ^= A[j][i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h
Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod{p}$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

```
int matInv(vector<vector<double>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;

    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j,i+1,n) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] -= f*A[i][k];
            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
        }
    }
}
```

```
rep(j,i+1,n) A[i][j] /= v;
rep(j,0,n) tmp[i][j] /= v;
A[i][i] = 1;
}

for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
}

rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
return n;
}
```

Tridiagonal.h
Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type
$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$
where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from
$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for diag[i] == 0 is needed.
Time: $\mathcal{O}(N)$

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.3 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)
"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
}
```

```
}
return res;
}
```

NumberTheoreticTransform.h

Description: $\text{ntt}(a)$ computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(\text{mod}-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n , reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$
"../number-theory/ModPow.h" ced03d, 34 lines

const ll mod = (119 << 23) + 1, root = 62; // = 998244353

```
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

464cf3, 16 lines

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
    }
    if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
    FST(a, 0); FST(b, 0);
```

```
rep(i,0,sz(a)) a[i] *= b[i];
FST(a, 1); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModPow.h b83e45, 8 lines

const ll mod = 1000000007; // faster if const

```
ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. $\text{modLog}(a,1,m)$ can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$
c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p
"ModPow.h" 19a793, 24 lines

```
ll sqrt(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

<pre>const int LIM = 1e6; bitset<LIM> isPrime; vi eratosthenes() { const int S = (int)round(sqrt(LIM)), R = LIM / 2; vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1)); vector<pii> cp; for (int i = 3; i <= S; i += 2) if (!sieve[i]) { cp.push_back({i, i * i / 2}); for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1; } for (int L = 1; L <= R; L += S) { array<bool, S> block{}; for (auto &[p, idx] : cp) for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1; rep(i,0,min(S, R - L)) if (!block[i]) pr.push_back((L + i) * 2 + 1); } for (int i : pr) isPrime[i] = 1; return pr; }</pre>	6b2912, 20 lines
---	------------------

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

<pre>"ModMullL.h" bool isPrime(ull n) { if (n < 2 n % 6 % 4 != 1) return (n 1) == 3; ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022}, s = __builtin_ctzll(n-1), d = n >> s; for (ull a : A) { // ^ count trailing zeroes ull p = modpow(a%n, d, n), i = s; while (p != 1 && p != n - 1 && a % n && i--) p = modmul(p, p, n); if (p != n-1 && i != s) return 0; } return 1; }</pre>	60dcd1, 12 lines
---	------------------

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

<pre>"ModMullL.h", "MillerRabin.h" ull pollard(ull n) { ull x = 0, y = 0, t = 30, prd = 2, i = 1, q; auto f = [&](ull x) { return modmul(x, x, n) + i; }; while (t++ % 40 __gcd(prd, n) == 1) { if (x == y) x = ++i, y = f(x); if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q; x = f(x), y = f(f(y)); } return __gcd(prd, n); } vector<ull> factor(ull n) { if (n == 1) return {}; if (isPrime(n)) return {n}; ull x = pollard(n); auto l = factor(x), r = factor(n / x); l.insert(l.end(), all(r)); return l; }</pre>	d8d98d, 18 lines
---	------------------

<pre>}</pre>	
PrimeFact.h Description: ... Time: ...	bcd355, 16 lines

<pre>const int N = 1e7; vector<int> spf(N); void prep(){ // modified sieve L(i, 2, N-1) { if(spf[i] == 0) { spf[i] = i; for(ll j = i * i; j <= N-1; j += i) { if(spf[j] == 0) spf[j] = i; } } } // prime factorize [if you need 1e9, use loop upto sqrt] vector<pll> prime_factorize(ll x) { vector<pll> ans; while(x!=1) { ll cur = spf[x]; ll cnt = 0; while(x%cur==0) cnt++, x/=cur; ans.push_back({cur, cnt}); } return ans; } }</pre>	
---	--

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod{b}$.

<pre>ll euclid(ll a, ll b, ll &x, ll &y) { if (!b) return x = 1, y = 0, a; ll d = euclid(b, a % b, y, x); return y -= a/b * x, d; }</pre>	33ba8f, 5 lines
---	-----------------

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

<pre>"euclid.h" ll crt(ll a, ll m, ll b, ll n) { if (n > m) swap(a, b), swap(m, n); ll x, y, g = euclid(m, n, x, y); assert((a - b) % g == 0); // else no solution x = (b - a) % n * x % n / g * m + a; return x < 0 ? x + m*n/g : x; }</pre>	04d93a, 7 lines
---	-----------------

5.3.1 Bézout's identity

For $a \neq 0, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1-1)p_1^{k_1-1} \dots (p_r-1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.

$\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p} \forall a$.

<pre>const int LIM = 5000000; int phi[LIM]; void calculatePhi() { rep(i,0,LIM) phi[i] = i&1 ? i : i/2; for (int i = 3; i < LIM; i += 2) if(phi[i] == i) for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i; }</pre>	cf7d6d, 8 lines
---	-----------------

5.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.5 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.6 Estimates

$\sum_{d|n} d = \mathcal{O}(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.7 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13				14	15	16	17
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h
Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
        use |= 1 << x;
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n\in S} \frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g\in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

<i>n</i>	0	1	2	3	4	5	6	7	8	9	20	50	100
<i>p</i> (<i>n</i>)	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

multinomial.h
Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$.
a0a312, 5 lines

```
ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
 $B[0,\dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^\infty f(i) &= \int_m^\infty f(x)dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$\begin{aligned} c(n,k) &= c(n-1,k-1) + (n-1)c(n-1,k), \quad c(0,0) = 1 \\ \sum_{k=0}^n c(n,k) x^k &= x(x+1) \dots (x+n-1) \end{aligned}$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
 $c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

Dijkstra.h

Description: ...

Time: ...

bbab71, 15 lines

vector<ll> dijkstra(ll start, vector<vector<array<ll, 2>>> &gr, int n){ vector<ll> d(n); For(i, 1, n) d[i] = inf; d[start] = 0; bool vis[n+1] = {0}; priority_queue<array<ll, 2>, vector<array<ll, 2>>, greater<array<ll, 2>>> pq; pq.push({0LL, start}); while(!pq.empty()){ auto [dist, node] = pq.top(); pq.pop(); if(vis[node]) continue; vis[node] = 1; for(auto [nextNode, weight] : gr[node]){ if(weight + dist < d[nextNode]){ d[nextNode] = weight + dist; pq.push({d[nextNode], nextNode}); }}} return d; }

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get $\text{dist} = \text{inf}$; nodes reachable through negative-weight cycles get $\text{dist} = -\text{inf}$. Assumes $V^2 \max |w_i| < \sim 2^{63}$.

Time: $\mathcal{O}(VE)$

830a8f, 23 lines

const ll inf = LLONG_MAX; struct Ed { int a, b, w, s() { return a < b ? a : -a; }}; struct Node { ll dist = inf; int prev = -1; }; void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) { nodes[s].dist = 0; sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); }); int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices rep(i,0,lim) for (Ed ed : eds) { Node cur = nodes[ed.a], &dest = nodes[ed.b]; if (abs(cur.dist) == inf) continue; ll d = cur.dist + ed.w; if (d < dest.dist) { dest.prev = ed.a; dest.dist = (i < lim-1 ? d : -inf); } } rep(i,0,lim) for (Ed e : eds) { if (nodes[e.a].dist == -inf) nodes[e.b].dist = -inf; } }

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j] = \text{inf}$ if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or $-\text{inf}$ if the path goes through a negative-weight cycle.

Time: $\mathcal{O}(N^3)$

531245, 12 lines

const ll inf = 1LL << 62; void floydWarshall(vector<vector<ll>>& m) { int n = sz(m); rep(i,0,n) m[i][i] = min(m[i][i], 0LL); rep(k,0,n) rep(i,0,n) rep(j,0,n) if (m[i][k] != inf && m[k][j] != inf) { auto newDist = max(m[i][k] + m[k][j], -inf); m[i][j] = min(m[i][j], newDist); } rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.

Time: $\mathcal{O}(|V| + |E|)$

d678d8, 8 lines

vi topoSort(const vector<vi>& gr) { vi indeg(sz(gr)), q; for (auto& li : gr) for (int x : li) indeg[x]++; rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i); rep(j,0,sz(q)) for (int x : gr[q[j]]) if (--indeg[x] == 0) q.push_back(x); return q; }

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(V^2\sqrt{E})$

0ae1d4, 48 lines

struct PushRelabel { struct Edge { int dest, back; ll f, c; }; vector<vector<Edge>> g; vector<ll> ec; vector<Edge*> cur; vector<vi> hs; vi H; PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {} void addEdge(int s, int t, ll cap, ll rcap=0) { if (s == t) return; g[s].push_back({t, sz(g[t]), 0, cap}); g[t].push_back({s, sz(g[s])-1, 0, rcap}); } void addFlow(Edge& e, ll f) { Edge &back = g[e.dest][e.back]; if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest); e.f += f; e.c -= f; ec[e.dest] += f; back.f -= f; back.c += f; ec[back.dest] -= f; } ll calc(int s, int t) { int v = sz(g); H[s] = v; ec[t] = 1; vi co(2*v); co[0] = v-1; rep(i,0,v) cur[i] = g[i].data(); for (Edge& e : g[s]) addFlow(e, e.c); for (int hi = 0;;) { while (hs[hi].empty()) if (!hi--) return -ec[s]; int u = hs[hi].back(); hs[hi].pop_back(); while (ec[u] > 0) // discharge u if (cur[u] == g[u].data() + sz(g[u])) { H[u] = 1e9; for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1) H[u] = H[e.dest]+1, cur[u] = &e; if (++co[H[u]], !--co[hi] && hi < v) rep(i,0,v) if (hi < H[i] && H[i] < v) --co[H[i]], H[i] = v + 1; hi = H[u]; } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

58385b, 79 lines

#include <bits/extc++.h> const ll INF = numeric_limits<ll>::max() / 4; struct MCMF { struct edge { int from, to, rev; ll cap, cost, flow; }; int N; vector<vector<edge>> ed; vi seen; vector<ll> dist, pi; vector<edge*> par; MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {} void addEdge(int from, int to, ll cap, ll cost) { if (from == to) return; ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 }); ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 }); } void path(int s) { fill(all(seen), 0); fill(all(dist), INF); dist[s] = 0; ll di; __gnu_pbds::priority_queue<pair<ll, int>> q; vector<decltype(q)::point_iterator> its(N); q.push({ 0, s }); while (!q.empty()) { s = q.top().second; q.pop(); seen[s] = 1; di = dist[s] + pi[s]; for (edge& e : ed[s]) if (!seen[e.to]) { ll val = di - pi[e.to] + e.cost; if (e.cap - e.flow > 0 && val < dist[e.to]) { dist[e.to] = val; par[e.to] = &e; if (its[e.to] == q.end()) its[e.to] = q.push({ -dist[e.to], e.to }); } else q.modify(its[e.to], { -dist[e.to], e.to }); } } rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF); }

```
totflow += fl;
for (edge* x = par[t]; x; x = par[x->from]) {
    x->flow += fl;
    ed[x->to][x->rev].flow -= fl;
}
}
rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $O(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

482fe0, 36 lines

```
template<class T> T edmondsKarp(vector<unordered_map<int, T>>&
    graph, int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vi par(sz(graph)), q = par;

    for (;;) {
        fill(all(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        rep(i,0,ptr) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
    }
out:
    T inc = numeric_limits<T>::max();
    for (int y = sink; y != source; y = par[y])
        inc = min(inc, graph[par[y]][y]);

    flow += inc;
    for (int y = sink; y != source; y = par[y]) {
        int p = par[y];
        if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
        graph[y][p] += inc;
    }
}
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i,0,n) co[i] = {i};
    rep(ph,1,n) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i,0,n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i,0,n) mat[s][i] += mat[t][i];
        rep(i,0,n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    rep(i,1,N) {
        PushRelabel D(N); // Dinic also works
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j,i+1,N)
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}
```

7.3 Matching

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); hopcroftKarp(g, btoa);
Time: $\mathcal{O}(\sqrt{VE})$

f612e4, 42 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
    if (A[a] != L) return 0;
    A[a] = -1;
    for (int b : g[a]) if (B[b] == L + 1) {
        B[b] = 0;
        if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
```

```
        return btoa[b] = a, 1;
    }
    return 0;
}

int hopcroftKarp(vector<vi>& g, vi& btoa) {
    int res = 0;
    vi A(g.size()), B(btoa.size()), cur, next;
    for (;;) {
        fill(all(A), 0);
        fill(all(B), 0);
        cur.clear();
        for (int a : btoa) if (a != -1) A[a] = -1;
        rep(a,0,sz(g)) if (A[a] == 0) cur.push_back(a);
        for (int lay = 1;; lay++) {
            bool islast = 0;
            next.clear();
            for (int a : cur) for (int b : g[a]) {
                if (btoa[b] == -1) {
                    B[b] = lay;
                    islast = 1;
                }
                else if (btoa[b] != a && !B[b]) {
                    B[b] = lay;
                    next.push_back(btoa[b]);
                }
            }
            if (islast) break;
            if (next.empty()) return res;
            for (int a : next) A[a] = lay;
            cur.swap(next);
        }
        rep(a,0,sz(g))
            res += dfs(a, 0, g, btoa, A, B);
    }
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); dfsMatching(g, btoa);
Time: $\mathcal{O}(VE)$

522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}

int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i,0,sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h"	da4196, 20 lines
<pre>vi cover(vector<vi>& g, int n, int m) { vi match(m, -1); int res = dfsMatching(g, match); vector<bool> lfound(n, true), seen(m); for (int it : match) if (it != -1) lfound[it] = false; vi q, cover; rep(i,0,n) if (lfound[i]) q.push_back(i); while (!q.empty()) { int i = q.back(); q.pop_back(); lfound[i] = 1; for (int e : g[i]) if (!seen[e] && match[e] != -1) { seen[e] = true; q.push_back(match[e]); } } rep(i,0,n) if (!lfound[i]) cover.push_back(i); rep(i,0,m) if (seen[i]) cover.push_back(n+i); assert(sz(cover) == res); return cover; }</pre>	

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

	1e0fe9, 31 lines
<pre>pair<int, vi> hungarian(const vector<vi> &a) { if (a.empty()) return {0, {}}; int n = sz(a) + 1, m = sz(a[0]) + 1; vi u(n), v(m), p(m), ans(n - 1); rep(i,1,n) { p[0] = i; int j0 = 0; // add "dummy" worker 0 vi dist(m, INT_MAX), pre(m, -1); vector<bool> done(m + 1); do { // dijkstra done[j0] = true; int i0 = p[j0], j1, delta = INT_MAX; rep(j,1,m) if (!done[j]) { auto cur = a[i0 - 1][j - 1] - u[i0] - v[j]; if (cur < dist[j]) dist[j] = cur, pre[j] = j0; if (dist[j] < delta) delta = dist[j], j1 = j; } rep(j,0,m) { if (done[j]) u[p[j]] += delta, v[j] -= delta; else dist[j] -= delta; } j0 = j1; } while (p[j0]); while (j0) { // update alternating path int j1 = pre[j0]; p[j0] = p[j1], j0 = j1; } } rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1; return {-v[0], ans}; // min cost }</pre>	

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod .
Time: $\mathcal{O}(N^3)$

"../numerical/MatrixInverse-mod.h"	cb1912, 40 lines
<pre>vector<pii> generalMatching(int N, vector<pii>& ed) { vector<vector<ll>> mat(N, vector<ll>(N)), A; for (pii pa : ed) { int a = pa.first, b = pa.second, r = rand() % mod; mat[a][b] = r, mat[b][a] = (mod - r) % mod; } int r = matInv(A = mat), M = 2*N - r, fi, fj; assert(r % 2 == 0); if (M != N) do { mat.resize(M, vector<ll>(M)); rep(i,0,N) { mat[i].resize(M); rep(j,N,M) { int r = rand() % mod; mat[i][j] = r, mat[j][i] = (mod - r) % mod; } } } while (matInv(A = mat) != M); vi has(M, 1); vector<pii> ret; rep(it,0,M/2) { rep(i,0,M) if (has[i]) rep(j,i+1,M) if (A[i][j] && mat[i][j]) { fi = i; fj = j; goto done; } assert(0); done: if (fj < N) ret.emplace_back(fi, fj); has[fi] = has[fj] = 0; rep(sw,0,2) { ll a = modpow(A[fi][fj], mod-2); rep(i,0,M) if (has[i] && A[i][fj]) { ll b = A[i][fj] * a % mod; rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod; } swap(fi,fj); } } return ret; }</pre>	

7.4 DFS algorithms

SCC.h

Description: ...

Time: ...

	fib591, 23 lines
<pre>vvll gr(N), revgr(N); vector<bool> vis(N); stack<ll> st; void dfs1(ll node) { vis[node] = 1; for (ll ch : gr[node]) { if (!vis[ch]) dfs1(ch); } st.push(node); } void dfs2(ll node, vll &comp) { vis[node] = 1; comp.push_back(node); for (ll ch : revgr[node]) { if (!vis[ch]) dfs2(ch, comp); } } vvll kosaraju() { // returns comps in topological order vvll comps; L(i, 1, n) if (!vis[i]) dfs1(i); L(i, 1, n) vis[i] = 0; while (st.size()) { ll now = st.top(); st.pop(); if (!vis[now]) {</pre>	

	vll comp; dfs2(now, comp); comps.push_back(comp);
	}
	} return comps;
	}

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps[&] (const vi& edgelist) {...});
Time: $\mathcal{O}(E + V)$

	c6b7c7, 32 lines
<pre>vi num, st; vector<vector<pii>> ed; int Time; template<class F> int dfs(int at, int par, F& f) { int me = num[at] = ++Time, top = me; for (auto [y, e] : ed[at]) if (e != par) { if (num[y]) { top = min(top, num[y]); if (num[y] < me) st.push_back(e); } else { int si = sz(st); int up = dfs(y, e, f); top = min(top, up); if (up == me) { st.push_back(e); f(vi(st.begin() + si, st.end())); st.resize(si); } else if (up < me) st.push_back(e); else { /* e is a bridge */ } } } return top; }</pre>	

<pre>template<class F> void bicomps(F f) { num.assign(sz(ed), 0); rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f); }</pre>

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

	484741, 56 lines
<pre>// variables are zero indexed struct twosat{ int n; vector<vector<int>> g,gt;</pre>	

```
vector<bool> vis,res;
vector<int> comp;
stack<int> ts;
twosat(int vars=0){
    n=vars<<1; g.resize(n); gt.resize(n); }

//if you want to force variable a to be true : addOR (a,1,a,1)
;
//if you want to force variable a to be false : addOR (a,0,a,0);

// adds an implication
void _add(int a,bool sign_a,int b,bool sign_b){
    a+=a*(sign_a^1);
    b+=b*(sign_b^1);
    g[a].push_back(b); gt[b].push_back(a); }

//(x_a or (not x_b)-> sign_a=1,sign_b=0
void addOR(int a,bool sign_a,int b,bool sign_b){
    a+=a*(sign_a^1);
    b+=b*(sign_b^1);
    g[a^1].push_back(b); // !a => b
    g[b^1].push_back(a); // !b => a
    gt[b].push_back(a^1);
    gt[a].push_back(b^1); }

//(!x_a xor !x_b)-> sign_a=0, sign_b=0
void addXOR(int a,bool sign_a,int b,bool sign_b){
    addOR(a,sign_a,b,sign_b);
    addOR(a,!sign_a,b,!sign_b); }

void dfs1(int u){
    vis[u]=true;
    for(int v:g[u] if(!vis[v]) dfs1(v);
    ts.push(u); }

void dfs2(int u,int c){
    comp[u]=c;
    for(int v:gt[u] if(comp[v]==-1) dfs2(v,c); }

bool run(){ // returns true if possible, else false
    vis.resize(n,false);
    for(int i=0;i<n;++i) if(!vis[i]) dfs1(i);
    int scc=0;
    comp.resize(n,-1);
    while(!ts.empty()){
        int u=ts.top(); ts.pop();
        if(comp[u]==-1) dfs2(u,scc++); }
    res.resize(n/2);
    for(int i=0;i<n;i+=2){
        if(comp[i]==comp[i+1]) return false;
        res[i/2]=(comp[i]>comp[i+1]); }
    return true; }
};
```

780b64, 15 lines

EulerWalk.h
Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

```
if (it == end){ ret.push_back(x); s.pop_back(); continue; }
tie(y, e) = gr[x][it++];
if (!eu[e]) {
    D[x]--, D[y]++;
    eu[e] = 1; s.push_back(y);
}
for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
return {ret.rbegin(), ret.rend()};
}
```

Bridges.h
Description: ...
Time: ...

98ffe3, 36 lines

```
struct BridgesAndCuts{
    int n,timer=0;
    vector<vector<int>>> g;
    vector<int> tin,low;
    vector<char> vis,is_cut;
    vector<pair<int,int>>> bridges;

    BridgesAndCuts(int n=0):n(n),g(n),tin(n,-1),
        low(n,-1),vis(n,0),is_cut(n,0){}

    void addEdge(int u,int v){ g[u].push_back(v); g[v].push_back(u)
        }; }

    void run(){
        timer=0;
        fill(tin.begin(),tin.end(),-1);
        fill(low.begin(),low.end(),-1);
        fill(vis.begin(),vis.end(),0);
        fill(is_cut.begin(),is_cut.end(),0);
        bridges.clear();
        for(int v=0;v<n;++v) if(!vis[v]) dfs(v,-1); }

private:
    void dfs(int v,int p){
        vis[v]=1;
        tin[v]=low[v]=timer++;
        int children=0;
        for(int to:g[v]){
            if(to==p) continue;
            if(vis[to]){ low[v]=min(low[v],tin[to]); }
            else{
                ++children; dfs(to,v);
                low[v]=min(low[v],low[to]);
                if(low[to]>tin[v]) bridges.emplace_back(min(v,to),max(v,to)
                    );
                if(p!=-1&&low[to]>=tin[v]) is_cut[v]=1; } }
        if(p==-1&&children>1) is_cut[v]=1; }
};
```

7.5 Coloring
EdgeColoring.h
Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
```

```
loc.assign(ncols, 0);
int at = u, end = u, d, c = free[u], ind = 0, i = 0;
while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
    loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
cc[loc[d]] = c;
for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
    swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
while (adj[fan[i]][d] != -1) {
    int left = fan[i], right = fan[++i], e = cc[i];
    adj[u][e] = left;
    adj[left][e] = u;
    adj[right][e] = -1;
    free[right] = e;
}
adj[u][d] = fan[i];
adj[fan[i]][d] = u;
for (int y : {fan[0], u, end})
    for (int& z = free[y] = 0; adj[y][z] != -1; z++);
}
rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
return ret;
}
```

7.6 Heuristics
MaximalCliques.h
Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.
Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
    if (!P.any()) { if (!X.any()) f(R); return; }
    auto q = (P | X)._Find_first();
    auto cands = P & ~eds[q];
    rep(i,0,sz(eds)) if (cands[i]) {
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

MaximumClique.h
Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
```

```
S[lev] += S[lev - 1] - old[lev];
old[lev] = S[lev - 1];
while (sz(R)) {
    if (sz(q) + R.back().d <= sz(qmax)) return;
    q.push_back(R.back().i);
    vv T;
    for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
    if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
            int k = 1;
            auto f = [&](int i) { return e[v.i][i]; };
            while (any_of(all(C[k]), f)) k++;
            if (k > mxk) mxk = k, C[mxk + 1].clear();
            if (k < mnk) T[j++].i = v.i;
            C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
            T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
    } else if (sz(q) > sz(qmax)) qmax = q;
    q.pop_back(), R.pop_back();
}
}
vi maxClique() { init(V), expand(V); return qmax; }
Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
}
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

```
vector<vi> treeJump(vi& P){
    int on = 1, d = 1;
    while(on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    rep(i,1,d) rep(j,0,sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
    rep(i,0,sz(tbl))
        if(steps&(1<<i)) nod = tbl[i][nod];
    return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}
```

```
}

LCA.h
Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time:  $\mathcal{O}(N \log N + Q)$ 
"/data-structures/RMQ.h"
0f62fb, 21 lines

struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};

CompressTree.h
Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most  $|S| - 1$ ) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time:  $\mathcal{O}(|S| \log |S|)$ 
"/LCA.h"
9775a0, 21 lines

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i,0,m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i,0,sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i,0,sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}

HLD.h
Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most  $\log(n)$  light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS.EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
```

```
Time:  $\mathcal{O}((\log N)^2)$ 
"/data-structures/LazySegmentTree.h"
cb0b6c, 49 lines

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    Lseg *tree;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N),pos(N),tree(new Lseg(0, N)){ dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (; v = par[rt[v]]) {
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree->add(l, r, val); });
    }
    void modifySubtree(int v, ll val) {
        tree->add(pos[v] + VALS_EDGES, pos[v] + siz[v], val);
    }
    int queryPath(int u, int v) { // Modify depending on problem
        int res = -1e9;
        process(u, v, [&](int l, int r) {
            res = max(res, (tree->query(l, r)).mx );
        });
        return res;
    }
    int querySubtree(int v) { // modifySubtree is similar
        return (tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v])).mx;
    }
};
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

```
0fb462, 90 lines

struct Node { // Splay tree. Root's pp contains tree's parent.
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    Node() { c[0] = c[1] = 0; fix(); }
    void fix() {
        if (c[0]) c[0]->p = this;
        if (c[1]) c[1]->p = this;
        // (+ update sum of subtree elements etc. if wanted)
    }
    void pushFlip() {
```

```

    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
}
int up() { return p ? p->c[1] == this : -1; }
void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
        x->c[h] = y->c[h ^ 1];
        y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
}
void splay() {
    for (pushFlip(); p; ) {
        if (p->p) p->p->pushFlip();
        p->pushFlip(); pushFlip();
        int c1 = up(), c2 = p->up();
        if (c2 == -1) p->rot(c1, 2);
        else p->p->rot(c2, c1 != c2);
    }
}
Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
}
};

struct LinkCut {
    vector<Node> node;
    LinkCut(int N) : node(N) {}

    void link(int u, int v) { // add an edge (u, v)
        assert(!connected(u, v));
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) { // remove an edge (u, v)
        Node *x = &node[u], *top = &node[v];
        makeRoot(top); x->splay();
        assert(top == (x->pp ? x->c[0]));
        if (x->pp) x->pp = 0;
        else {
            x->c[0] = top->p = 0;
            x->fix();
        }
    }
    bool connected(int u, int v) { // are u, v in the same tree?
        Node* nu = access(&node[u])->first();
        return nu == access(&node[v])->first();
    }
    void makeRoot(Node* u) {
        access(u);
        u->splay();
        if (u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0;
            u->fix();
        }
    }
}

```

```

Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
        pp->splay(); u->pp = 0;
        if (pp->c[1]) {
            pp->c[1]->p = 0; pp->c[1]->pp = pp; }
        pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
}
};

```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 59 lines

```

struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

```

```

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n, par(n));
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1,-1}), comp;
    deque<tuple<int, int, vector<Edge>>> cyps;
    rep(s,0,n) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1,{};};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cyps.push_front({u, time, {Q[qi], &Q[end]}});
            }
        }
        rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
    }
}

```

for (auto& [u,t,comp] : cyps) { // restore sol (optional)

```

uf.rollback(t);
Edge inEdge = in[u];
for (auto& e : comp) in[uf.find(e.b)] = e;
in[uf.find(inEdge.b)] = inEdge;
}
rep(i,0,n) par[i] = in[i].a;
return {res, par};}

```

TreeFlatten.h

Description: ...

Time: ...

b731c2, 14 lines

```

vll in(N), out(N); ll timer = -1;
void euler_tour(ll node, ll par) {
    in[node] = ++timer;
    for (ll ch : gr[node]) {
        if (ch == par) continue;
        euler_tour(ch, node);
    }
    out[node] = timer;
}
// usage :
euler_tour(1, 0);
vll flat_tree(n);
L(i, 1, n) flat_tree[in[i]] = a[i];
sumSeg seg(flat_tree);

```

CentDecom.h

Description: ...

Time: ...

b47437, 35 lines

```

ll n,m,x,y,z,q,k,u,v,w;
vector<int> gr[N];
int sz[N];
int tot,done[N],cenpar[N];

```

```

void calc_sz(int node,int p){
    tot++; sz[node]=1;
    for(auto ch:gr[node]){
        if(ch==p||done[ch]) continue;
        calc_sz(ch,node); sz[node]+=sz[ch]; } }

```

```

int find_cen(int node,int p){ // find centroid
    for(auto ch:gr[node]){
        if(ch==p||done[ch]) continue;
        else if(sz[ch]>tot/2) return find_cen(ch,node); }
    return node; }

```

```

void decompose(int node,int p){ // find centroid of subtree
    tot=0; calc_sz(node,p);
    int cen=find_cen(node,p);
    cenpar[cen]=p; done[cen]=1;
    for(auto ch:gr[cen]){
        if(ch==p||done[ch]) continue;
        decompose(ch,cen); } }

```

vll cen_tree[N]; // centroid tree

```

int form_cen_tree(){ // form graph, return root(centroid)
    decompose(1,0); int root;
    L(i,1,n){
        if(cenpar[i]==0) root=i;
        if(cenpar[i]!=0){
            cen_tree[i].push_back(cenpar[i]);
            cen_tree[cenpar[i]].push_back(i); } }
    return root; }

```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \rightarrow b \in G$, do `mat[a][b]--`, `mat[b][b]++` (and `mat[b][a]--`, `mat[a][a]++` if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Bruns-Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

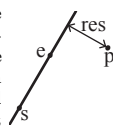
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")"; }
};
```

lineDistance.h

Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



Point.h

60bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double) (b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

Point.h

5c88f4, 6 lines

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

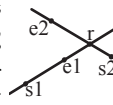
Point.h, "OnSegment.h"

9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h

Description: If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

Point.h

a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

Point.h

3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

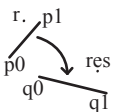
Point.h

c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



Point.h

03a306, 6 lines

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

Point.h

0f0602, 35 lines

```
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
    }
```

```
    return y < 0 || (y == 0 && x < 0);
}
Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
Angle t180() const { return {-x, -y, t + half()}; }
Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (1l)b.x) <
           make_tuple(b.t, b.half(), a.x * (1l)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180()) ?
           make_pair(a, b) : make_pair(b, a.t360());
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"	84d6d3, 11 lines
-----------	------------------

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
           p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"	b0153d, 13 lines
-----------	------------------

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

"../content/geometry/Point.h"	19add1, 19 lines
-------------------------------	------------------

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = q + d * (t-1);
        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

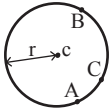
circumcircle.h

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h"	1caa3a, 9 lines
-----------	-----------------

```
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()* (C-B).dist()* (A-C).dist() /
           abs((B-A).cross(C-A)) / 2;
}
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```



MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

"circumcircle.h"	09dd0a, 17 lines
------------------	------------------

```
pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};

bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"	2bf504, 11 lines
---	------------------

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	f12300, 6 lines
-----------	-----------------

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

"Point.h"	9706dc, 9 lines
-----------	-----------------

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h

Description:

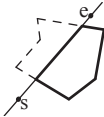
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;

p = polygonCut(p, P(0,0), P(1,0));

"Point.h"	d07181, 13 lines
-----------	------------------

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;
    rep(i,0,sz(poly)) {
        P cur = poly[i], prev = i ? poly[i-1] : poly.back();
        auto a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a < 0) != (b < 0))
            res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.push_back(cur);
    }
    return res;
}
```



ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $O(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $O(n)$

"Point.h" c571b8, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res(0, {S[0], S[0]});
    rep(i, 0, j)
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $O(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $O(\log n)$

"Point.h" 7cf45b, 39 lines

```
#define cmp(i, j) sgn(dir.perp().cross(poly[i] - poly[j] % n))
#define cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}
```

```
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $O(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d(1 + (ll)sqrt(ret.first), 0);
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(lo - p).dist2(), {lo, p}});
    }
```

```
S.insert(p);
}
return ret.second;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h" bac5b0, 63 lines

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();
```

```
bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }
```

```
struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;
```

```
T distance(const P& p) { // min squared distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x, y) - p).dist2();
}
```

```
Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
        x0 = min(x0, p.x); x1 = max(x1, p.x);
        y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
        // split on x if width >= height (not ideal...)
        sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
        // divide by taking half the array for each child (not
        // best performance with many duplicates in the middle)
        int half = sz(vp)/2;
        first = new Node({vp.begin(), vp.begin() + half});
        second = new Node({vp.begin() + half, vp.end()});
    }
}
```

```
struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}
```

```
pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
        // uncomment if we should not find the point itself:
        // if (p == node->pt) return {INF, P()};
        return make_pair((p - node->pt).dist2(), node->pt);
    }
```

```
Node *f = node->first, *s = node->second;
T bfirst = f->distance(p), bsec = s->distance(p);
if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);
```

```
// search closest side first, other side if needed
auto best = search(f, p);
if (bsec < best.first)
    best = min(best, search(s, p));
return best;
}
```

```
// find nearest point to a point, and its squared distance
// (requires an arbitrary operator< for Point)
pair<T, P> nearest(const P& p) {
    return search(root, p);
}
```

```

    }
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ... }, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h" eeefd5, 88 lines

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point
```

```
struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;
```

```
bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
```

```
Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}
```

```
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
```

```
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}
```

```
pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }
}
```

```
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
Q A, B, ra, rb;
int half = sz(s) / 2;
tie(ra, A) = rec({all(s) - half});
tie(B, rb) = rec({sz(s) - half + all(s)});
while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
Q base = connect(B->r(), A);
if (A->p == ra->p) ra = base->r();
if (B->p == rb->p) rb = base;
```

```
#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
```

FastDelaunay PolyhedronVolume Point3D 3dHull

```

while (circ(e->dir->F(), H(base), e->F())) { \
    Q t = e->dir; \
    splice(e, e->prev()); \
    splice(e->r(), e->r()->prev()); \
    e->o = H; H = e; e = t; \
}
for (;;) {
    DEL(LC, base->r(), o); DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
        base = connect(RC, base->r());
    else
        base = connect(base->r(), LC->r());
}
return { ra, rb };
}
```

```
vector<P> triangulate(vector<P> pts) {
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
    return pts;
}
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

```
8058ae, 32 lines
template<class T> struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(R p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0, pi]
```

```

double theta() const { return atan2(sqrt(x*x+y*y),z); }
P unit() const { return *this/(T)dist(); } //makes dist()==1
//returns unit vector normal to *this and p
P normal(P p) const { return cross(p).unit(); }
//returns point rotated 'angle' radians ccw around axis
P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
}
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

"Point3D.h" 5b45fc, 49 lines

```
typedef Point3D<double> P3;
```

```
struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};
```

```
struct F { P3 q; int a, b, c; };
```

```
vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f(q, i, j, k);
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);
```

```

    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j,0,nw) {
            F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c); C(a, c, b); C(b, c, a);
        }
    }
    for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
        A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
    return FS;
};
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

```
611f07, 8 lines
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Strings (9)

KMP.h

```
Time:  $\mathcal{O}(n)$ 
3ac13f, 29 lines
// for each position (0 based ***) of s, what is the best match
// of a suffix at that position with a prefix of s
vector<int> prefixFunction(const string& s){
    int n=s.size(); vector<int> pi(n); pi[0]=0;
    for(int i=1; i<n; ++i){
        int j = pi[i-1];
        while(j>0 and s[j] != s[i]) j = pi[j-1];
        if(s[j] == s[i]) j++;
        pi[i]=j;
    }
    return pi;
}

// if the automaton is built on string s, then :
// aut[j][ch] = if i'm scanning/constructing some string and rn
// the best match between prefix_of_s and
// suffix_of-MY_string
// is of LENGTH j, and the next scanned/put character is ch,
// then what would be the best matched LENGTH
vector<vector<int>> prefixAutomaton(const string& s){
    int m = s.size();
    vector<int> pi = prefixFunction(s);
    const int alph = 26;
    vector<vector<int>> aut(m+1, vector<int>(alph));
    for(int j = 0; j <= m; j++){
        for(int c=0; c<alph; c++){
            if(j<m && s[j] == char('a'+c)) aut[j][c] = j+1;
            else if(j == 0) aut[j][c] = 0;
            else aut[j][c] = aut[pi[j-1]][c];
        }
    }
    return aut;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

```
f71b39, 12 lines
vi Z(const string& s) {
    vi z(sz(s));
    int l = -1, r = -1;
    rep(i,l,sz(s)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
```

```
while (i + z[i] < sz(s) && s[i + z[i]] == s[z[i]])
    z[i]++;
if (i + z[i] > r)
    l = i, r = i + z[i];
}
return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = radius of longest even palindrome where central pair is (i-1, i), p[1][i] = radius of longest odd (center EXCLUDED).
Time: $\mathcal{O}(N)$

```
e7ad79, 13 lines
array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi,2> p = {vi(n+1), vi(n)};
    rep(z,0,2) for (int i=0, l=0, r=0; i < n; i++) {
        int t = r-i+!z;
        if (i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i - p[z][i], R = i + p[z][i] - !z;
        while (L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}
```

Trie.h

```
Description: ...
Time: ...
6db610, 70 lines
struct Trie{
    static const int ALPHABET=26;
    static const char BASE='a';
    struct Node{
        int next[ALPHABET];
        int ends_here_cnt,count; // passes this node
        Node() {
            fill(next,next+ALPHABET,-1);
            ends_here_cnt=0; count=0;
        }
    };
    vector<Node> nodes;
    Trie(){ nodes.emplace_back(); } // root node

    void insert(const string& s){
        int cur=0;
        for(char ch:s){
            int c = ch-BASE;
            if(nodes[cur].next[c] == -1){
                nodes[cur].next[c]=nodes.size(); nodes.emplace_back();
            }
            cur = nodes[cur].next[c]; nodes[cur].count++;
        }
        nodes[cur].ends_here_cnt++;
    }

    bool erase(const string& s){
        if(!search(s)) return false;
        int cur=0;
        for(char ch:s){
            int c=ch-BASE;
            cur=nodes[cur].next[c]; nodes[cur].count--;
        }
        nodes[cur].ends_here_cnt--;
        return true;
    }
}
```

```
bool search(const string& s)const{
    int cur=0;
    for(char ch:s){
        int c=ch-BASE;
        if(nodes[cur].next[c]==-1
            || nodes[nodes[cur].next[c]].count==0) return false;
        cur=nodes[cur].next[c];
    }
    return nodes[cur].ends_here_cnt>0;
}

bool starts_with(const string& prefix)const{
    int cur=0;
    for(char ch:prefix){
        int c=ch-BASE;
        if(nodes[cur].next[c]==-1
            || nodes[nodes[cur].next[c]].count==0) return false;
        cur=nodes[cur].next[c];
    }
    return true;
}

int count_prefix(const string& prefix)const{
    int cur=0;
    for(char ch:prefix){
        int c=ch-BASE;
        if(nodes[cur].next[c]==-1
            || nodes[nodes[cur].next[c]].count==0) return 0;
        cur=nodes[cur].next[c];
    }
    return nodes[cur].count;
}
};

Hashing.h
Description: Self-explanatory methods for string hashing.
2d2a67, 44 lines
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
    ull x; H(ull x=0) : x(x) {}
    H operator+(H o) { return x + o.x + (x + o.x < x); }
    H operator-(H o) { return *this + ~o.x; }
    H operator*(H o) { auto m = (__uint128_t)x * o.x;
        return H((ull)m + (ull)(m >> 64)); }
    ull get() const { return x + !~x; }
    bool operator==(H o) const { return get() == o.get(); }
    bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (11)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
    vector<H> ha, pw;
    HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
        pw[0] = 1;
        rep(i,0,sz(str))
            ha[i+1] = ha[i] * C + str[i],
            pw[i+1] = pw[i] * C;
    }
    H hashInterval(int a, int b) { // hash [a, b)
        return ha[b] - ha[a] * pw[b - a];
    }
};

vector<H> getHashes(string& str, int length) {
```

```
    if (sz(str) < length) return {};\n    H h = 0, pw = 1;\n    rep(i,0,length)\n        h = h * C + str[i], pw = pw * C;\n    vector<H> ret = {h};\n    rep(i,length,sz(str)) {\n        ret.push_back(h = h * C + str[i] - pw * str[i-length]);\n    }\n    return ret;\n}
```

```
H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is *i*'th in the sorted suffix array. The returned vector is of size *n* + 1, and sa[0] = *n*. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars. **Time:** $\mathcal{O}(n \log n)$

```
struct SuffixArray {\n    vi sa, lcp;\n    SuffixArray(string s, int lim=256) { // or vector<int>\n        s.push_back(0); int n = sz(s), k = 0, a, b;\n        vi x(all(s)), y(n), ws(max(n, lim));\n        sa = lcp = y, iota(all(sa), 0);\n        for (int j = 0, p = 0; p < n; j = max(1ll, j * 2), lim = p)\n            {\n                p = j, iota(all(y), n - j);\n                rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;\n                fill(all(ws), 0);\n                rep(i,0,n) ws[x[i]]++;\n                rep(i,1,lim) ws[i] += ws[i - 1];\n                for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];\n                swap(x, y), p = 1, x[sa[0]] = 0;\n                rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =\n                    (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;\n            }\n        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)\n            for (k && k--, j = sa[x[i] - 1];\n                s[i + k] == s[j + k]; k++);\n    }\n};
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(–, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. **Time:** construction takes $\mathcal{O}(26N)$, where *N* = sum of length of patterns. find(*x*) is $\mathcal{O}(N)$, where *N* = length of *x*. findAll is $\mathcal{O}(NM)$.

```
struct Aho{\n    static const int K=26; // alphabet size\n    struct Vertex{\n        int next[K],link,term_link; // suffix link, terminal link\n        bool output; // true if this node is end of some pattern\n        vector<int> ids; // pattern ids ending here\n        int len;\n        Vertex(){\n            fill(begin(next), end(next), -1);\n            link=-1; term_link=-1; output=false; len=0;\n        }\n    };\n};
```

SuffixArray AhoCorasick Eertree SuffixAutomaton

```
vector<Vertex> t = {Vertex()}; // root = state 0\n\nvoid add_string(const string& s,int id){\n    int v=0;\n    for(char ch:s){\n        int c=ch-’a’;\n        if(t[v].next[c] == -1){\n            t[v].next[c] = t.size(); t.emplace_back();\n        }\n        v=t[v].next[c];\n    }\n    t[v].output=true; t[v].ids.push_back(id);\n}
```

```
void build_automaton(){ // build the automaton T.T\n    queue<int> q;\n    t[0].link=0; t[0].term_link=0;\n    for(int c=0; c<K; c++){\n        int u=t[0].next[c];\n        if(u!=-1){ t[u].link=0; q.push(u); t[u].len=1; }\n        else t[0].next[c]=0;// missing edge from root loops to root\n    }\n    while(!q.empty()){ \n        int v=q.front(); q.pop();\n        for(int c=0; c<K; c++){ \n            int u=t[v].next[c];\n            if(u!=-1){ // compute suffix link\n                t[u].link = t[t[v].link].next[c];\n                t[u].len = t[v].len+1;\n                q.push(u);\n            }\n            else t[v].next[c] = t[t[v].link].next[c];\n        }\n        // compute terminal link\n        if(t[t[v].link].output) t[v].term_link = t[v].link;\n        else t[v].term_link = t[t[v].link].term_link;\n    }\n}\n\nvoid form_tree(){\n    gr.resize(t.size());\n    L(state,1,t.size()-1){\n        gr[t[state].term_link].push_back(state);\n    }\n}\n};
```

Eertree.h

Description: ...
Time: ...

```
sz - 2 // no. of distinct palindromes\nno. of unique palindromes is linear !!! may use with hashing
```

use cnt during construction. After extend(i):
last = node representing the longest palindrome ending at pos i
pt.t[pt.last].cnt; // no. of pals ending here, smaller included

```
use oc after calc.\nt[i].oc //no. of occurence of this exact pal[smaller excluded]\nsum(t[i].oc) // total no. of palindromic substrings
```

```
struct PalindromicTree {\n    struct node {\n        int nxt[26]; // transitions (adding char to both ends)\n        int len; // length of palindrome\n        int st, en; // start and end index in original string\n        int link; // suffix link (largest palindromic suffix)\n        ll cnt; // no. of palindromic suffixes ending here\n    };\n};
```

```
ll oc; // no. of occurrences of this palindrome\n};\nstring s;\nvector<node> t;\nint sz, last;\nPalindromicTree() {} \nPalindromicTree(string _s) {\n    s = _s;\n    int n = s.size();\n    t.clear();\n    t.resize(n + 9);\n    sz = 2, last = 2;\n    t[1].len = -1, t[1].link = 1;\n    t[2].len = 0, t[2].link = 1;\n}\nint extend(int pos){//returns 1 if it creates a new palindrome\n    int cur = last, curlen = 0;\n    int ch = s[pos] - ’a’;\n    while (1) {\n        curlen = t[cur].len;\n        if(pos-1-curlen >= 0 && s[pos-1-curlen] == s[pos]) break;\n        cur = t[cur].link;\n    }\n    if (t[cur].nxt[ch]) {\n        last = t[cur].nxt[ch];\n        t[last].oc++;\n        return 0;\n    }\n    sz++;\n    last = sz;\n    t[sz].oc = 1;\n    t[sz].len = t[cur].len + 2;\n    t[cur].nxt[ch] = sz;\n    t[sz].en = pos;\n    t[sz].st = pos - t[sz].len + 1;\n    if (t[sz].len == 1) {\n        t[sz].link = 2;\n        t[sz].cnt = 1;\n        return 1;\n    }\n    while (1) {\n        cur = t[cur].link;\n        curlen = t[cur].len;\n        if (pos-1-curlen >= 0 && s[pos-1-curlen] == s[pos]) {\n            t[sz].link = t[cur].nxt[ch];\n            break;\n        }\n    }\n    t[sz].cnt = 1 + t[t[sz].link].cnt;\n    return 1;\n}\n\nvoid calc_occurrences() {\n    for (int i = sz; i >= 3; i--) t[t[i].link].oc += t[i].oc;\n}\n};
```

SuffixAutomaton.h

Description: ...
Time: ...

```
struct SAM{\n    struct State{\n        int link, maxlen;\n        map<char,int> next;\n        State(){ link=-1; maxlen=0; }\n    };\n\n    vector<State> st;\n    vector<vector<int>> gr;
```

```
vector<ll> koybar;
vector<bool> was_terminal;
int last,sz;
ll total; // number of unique substrings

SAM(int n){
    st.resize(2*n); gr.resize(2*n); koybar.resize(2*n);
    was_terminal.resize(2*n);
    st[0]=State(); sz=1; last=0; total=0;
}

void extend(char c){
    int cur = sz++;
    st[cur].maxlen = st[last].maxlen+1;
    int p=last;
    while(p!=-1 && !st[p].next.count(c)){
        st[p].next[c]=cur; p=st[p].link;
    }
    if(p == -1){ st[cur].link=0; }
    else{
        int q = st[p].next[c];
        if(st[p].maxlen+1 == st[q].maxlen){ st[cur].link=q; }
        else{
            int clone=sz++;
            st[clone] = st[q];
            st[clone].maxlen = st[p].maxlen + 1;
            while(p!=-1 && st[p].next[c] == q){
                st[p].next[c]=clone; p=st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
        last=cur; was_terminal[last]=true;
        total+=st[cur].maxlen-st[st[cur].link].maxlen;
    }
}

void build_tree(){ // USE (sam.sz-1) when you need to visit all states
    L(i,1,sz-1) gr[st[i].link].push_back(i);
}

void guno(){ // count repetitions in the string for each node
    int cur=last;
    L(i,1,sz-1) koybar[i] = was_terminal[i];
    function<void(int)> dfs=[&](int node){
        for(int ch:gr[node])
            { dfs(ch); koybar[node] += koybar[ch]; }
    };
    dfs(0);
}

ll koyta(int i){ return st[i].maxlen-st[st[i].link].maxlen; }

// after adding the entire string, if you need terminal states
:
// start from last, go back by suffix links. all visited
states are terminal states
};
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

d07a42, 8 lines

```
int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    rep(b,0,N) rep(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break;}
    }
```

```
}
    return a;
}

SuffixTree.h
Description: Ukkonen's algorithm for online suffix tree construction. Each
node contains indices [l, r] into the string, and a list of child nodes. Suffixes
are given by traversals of this tree, joining [l, r] substrings. The root is 0 (has
l = -1, r = 0), non-existent children are -1. To get a complete tree, append
a dummy symbol - otherwise it may contain an incomplete path (still useful
for substring matching, though).
Time:  $\mathcal{O}(26N)$ 
```

aae0b8, 50 lines

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

    void ukkadd(int i, int c) { suff:
        if (r[v]<=q) {
            if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
                p[m++]=v; v=s[v]; q=r[v]; goto suff; }
            v=t[v][c]; q=l[v];
        }
        if (q==-1 || c==toi(a[q])) q++; else {
            l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
            p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
            l[v]=q; p[v]=m; t[p[m]][toi(a[l[m]])]=m;
            v=s[p[m]]; q=l[m];
            while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
            if (q==r[m]) s[m]=v; else s[m]=m+2;
            q=r[v]-(q-r[m]); m+=2; goto suff;
        }
    }

    SuffixTree(string a) : a(a) {
        fill(r,r+N,sz(a));
        memset(s, 0, sizeof s);
        memset(t, -1, sizeof t);
        fill(t[1],t[1]+ALPHA,0);
        s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
        rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
    }

    // example: find longest common substring (uses ALPHA = 28)
    pii best;
    int lcs(int node, int i1, int i2, int olen) {
        if (l[node] <= i1 && i1 < r[node]) return 1;
        if (l[node] <= i2 && i2 < r[node]) return 2;
        int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
        rep(c,0,ALPHA) if (t[node][c] != -1)
            mask |= lcs(t[node][c], i1, i2, len);
        if (mask == 3)
            best = max(best, {len, r[node] - len});
        return mask;
    }

    static pii LCS(string s, string t) {
        SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
        st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
        return st.best;
    }
};
```

BitTrie.h

Description: ...

Time: ...

01eb16, 53 lines

```
struct BitTrie{
```

```
struct Node{
    int child[2],cnt;//trie[0]=total numbers inserted in trie
    Node(){ child[0] = child[1] = -1; cnt=0; }
};
vector<Node> trie;
int BITS;

BitTrie(int maxBits = 30){ // 30 for numbers up to 1e9
    BITS = maxBits; trie.push_back(Node()); // root
}

void insert(int x){
    int node=0;
    trie[node].cnt++;
    for(int b=BITS; b>=0; b--){
        int bit = (x>>b)&1;
        if(trie[node].child[bit] == -1){
            trie[node].child[bit] = trie.size();
            trie.push_back(Node());
        }
        node = trie[node].child[bit]; trie[node].cnt++;
    }
}

void erase(int x){
    int node=0;
    trie[node].cnt--;
    for(int b=BITS; b>=0; b--){
        int bit = (x>>b)&1;
        int nxt=trie[node].child[bit];
        node=nxt; trie[node].cnt--;
    }
}

ll max_xor(ll x){ // find y from this trie that maximizes x^y
    if(trie[0].cnt == 0) return 0; // empty trie
    ll cur=0, y=0;
    R(bit, BITS, 0){
        int nextbit = getBit(x,bit); // at root, nextbit=30
        if(trie[cur].child[!nextbit] != -1
            and trie[trie[cur].child[!nextbit]].cnt){
            cur = trie[cur].child[!nextbit];
            if(!nextbit) y = setbit(y,bit);
        }
        else{ // has to exist, since tire is not empty
            cur = trie[cur].child[nextbit];
            if(nextbit) y = setbit(y,bit);
        }
    }
    return x^y;
}
};
```

Various (10)

10.1 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in [a,b] that maximizes f(i), assuming that f(a) < ... < f(i) ≥ ... ≥ f(b). To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).

Usage: int ind = ternSearch(0,n-1,[&](int i){return a[i];});

Time: $\mathcal{O}(\log(b-a))$

9155b4, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
```

```
while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
}
rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
return a;
}
```

LIS.h
Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i,0,sz(S)) {
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

Diophantine.h
Description: ...
Time: ...

```
// Extended Euclidean Algorithm
ll ext_gcd(ll a,ll b,ll& x,ll& y){
    if(b==0){ x=1,y=0; return a; }
    ll x1,y1,g=ext_gcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g; }
// Solve ax + by = c
// general solution : x + k*b/g, y - k*a/g
bool solve_diophantine(ll a,ll b,ll c,ll& x,ll& y,ll& g){
    g=ext_gcd(abs(a),abs(b),x,y);
    if(c%g!=0) return false;
    x*=c/g; y*=c/g;
    if(a<0)x=-x; if(b<0)y=-y; return true; }
```

```
bool solve_dioph_non_neg(ll a,ll b,ll c,ll& x,ll& y,ll& g){
    if(!solve_diophantine(a,b,c,x,y,g)) return false;
    ll dx=(b/g),dy=(a/g);
    if(x<0){
        ll k=(-x+abs(dx)-1)/abs(dx); // ceil
        if(dx<0)k=-k; x+=k*dx; y-=k*dy; }
    if(y<0){
        ll k=(-y+abs(dy)-1)/abs(dy);
        if(dy<0)k=-k; y+=k*dy; x-=k*dx; }
    return x>=0 and y>=0; }
```

10.2 Dynamic programming

SOSDP.h
Description: ...
Time: ...

```
// for problems like sum/count involving submaks/supermask,
// we use sos dp to avoid overcounting.
// x|y = x : y is submask of x
// x&y = x : y is supermask of x
// x&y = 0 : y is submask of ~x
```

```
// k = koyta bit max

void fwd1(vll& dp, int k){ // dp[x] = cnt of submask of x
    int N = 1ll<<k;
    L(bit,0,k-1){
        L(mask,0,N-1){
            if(isSet(mask,bit)) dp[mask]+=dp[resetbit(mask,bit)]; } } }
```

```
void bak1(vll& dp, int k){ // return from submask count to mask count
    int N = 1ll<<k;
    R(bit,19,0){
        R(mask,N-1,0){
            if(isSet(mask,bit)) dp[mask]-=dp[resetbit(mask,bit)]; } } }
```

```
void fwd2(vll& dp, int k){ // dp[x] = cnt of supermask of x
    int N = 1ll<<k;
    L(bit,0,19){
        R(mask,N-1,0){
            if(isSet(mask,bit)) dp[resetbit(mask,bit)]+=dp[mask]; } } }
```

```
void bak2(vll& dp, int k){ // return from supermask count to mask count
    int N = 1ll<<k;
    R(bit,19,0){
        L(mask,0,N-1){
            if(isSet(mask,bit)) dp[resetbit(mask,bit)]-=dp[mask]; } } }
```

DigitDP.h
Description: ...
Time: ...

```
ll n,m,k,q;
ll a[N];
ll dp[][11][2][2]; // put max len of string + 1
```

```
bool is_valid(){ return 1; } // for even sum, return sum % 2 == 0;
```

```
ll cnt(string s,ll pos,ll prev,bool started,bool tight){
    if(pos==Size(s)) return is_valid(); // number of such numbers = 1
    if(dp[pos][prev][started][tight]!=-1) return dp[pos][prev][started][tight];
    ll ans=0;
    ll digMax=tight?s[pos]-'0':9;
    for(ll dig=0;dig<=digMax;dig++){
        // ***
        if(started and dig==prev) continue; // no repetition
        bool startedNext=started?1:dig!=0; // started || (dig != 0)
        bool tightNext=tight?(dig==digMax):0; // tight && (dig == digMax)
        ans+=cnt(s,pos+1,dig,startedNext,tightNext); }
    return dp[pos][prev][started][tight]=ans; }
```

```
// usage:
string s1 = to_string(n1 - 1);
memset(dp, -1, sizeof(dp));
ll ans1; if(n1-1 < 0) ans1 = 0;
// 10 is a dummy value for prev.
else ans1 = cnt(s1, 0, 10, 0, 1);
// similarly, find ans2, and output : ans2-ans1
```

DivideAndConquerDP.h
Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$.
Time: $\mathcal{O}((N + (hi - lo)) \log N)$

```
struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
```

```
void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
        best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
}
void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

BumpAllocator.h
Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

```
745db2, 8 lines
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
    static size_t i = sizeof buf;
    assert(s < i);
    return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

submasks.h
Description: ...
Time: ...

```
b0dc54, 2 lines
for (int mask = 0; mask < (1<<n); mask++)
    for(int i = mask; i; i = (i-1) & mask)
```

int128.h
Description: ...
Time: ...

```
using i128=__int128_t;
i128 read(){
    string s; cin>>s; bool neg=false; size_t i=0;
    if(s[0]=='-'){ neg=true; i=1; } i128 mag=0;
    for(;i<s.size();++i){ mag=mag*10+(unsigned)(s[i]-'0'); }
    return neg?-mag:mag; }
void print(i128 x){
    if(x==0){cout<<0<<nl;return;}
    bool neg=x<0; if(neg)x=-x;
    string s;while(x){ int d=(int)(x%10);
        s.push_back(char('0'+d)); x/=10; }
    if(neg)s.push_back('-'); reverse(s.begin(),s.end());
    cout<<s<<nl; }
bool cmp(i128 x,i128 y){ return x>y; } // for sorting
```

FloorCeilDiv.h
Description: ...
Time: ...

```
d77b32, 10 lines
ll floor_div(ll a, ll b) {
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return a / b;
    return (a - b + 1) / b;
}
ll ceil_div(ll a, ll b) {
```

```
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return (a + b - 1) / b;
    return a / b;
}
```

LinearSieve.h

Description: ...
Time: ... 84f2ca, 34 lines

```
const int N = 1000000;
vector<int> primes;
int spf[N + 1]; int phi[N + 1]; int mu[N + 1];
bool is_composite[N + 1];
// int func[MAXN], cnt[MAXN]; // Extension

void linear_sieve(int n) {
    std::fill(is_composite, is_composite + n, false);
    phi[1] = 1; mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!is_composite[i]) {
            primes.push_back(i);
            spf[i] = i; phi[i] = i - 1; mu[i] = -1;
        }
        for (int p : primes) {
            if (i * 1LL * p > n) break;
            is_composite[i * p] = true;
            spf[i * p] = p;

            if (i % p == 0) {
                phi[i * p] = phi[i] * p;
                mu[i * p] = 0;
                // func[i*prime[j]] = func[i]/cnt[i] * (cnt[i]+1);
                // cnt[i * prime[j]] = cnt[i] + 1;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
                mu[i * p] = -mu[i];
                // func[i * prime[j]] = func[i] * func[prime[j]];
                // cnt[i * prime[j]] = 1;
            }
        }
    }
}
```

XorBasis.h

Description: ...
Time: ... 5be624, 59 lines

```
struct XorBasis {
    static const int B = 60; // for 64-bit numbers
    long long b[B]; int sz;
    XorBasis() {
        memset(b, 0, sizeof b); sz = 0;
    }
    // Insert number into basis
    void insert(long long x) {
        for (int i = B-1; i >= 0; i--) {
            if (!(x >> i & 1)) continue;
            if (b[i] x ^= b[i]; // eliminate bit
            else { // new basis vector
                b[i] = x; sz++; return;
            }
        }
    }
    // Check if some subset has xor = x
    bool can(long long x) {
        for(int i = B-1; i>=0; i--) x = min(x, x ^ b[i]);
        return x == 0;
    }
    // Maximum possible xor with optional start value
    long long max_xor(long long x = 0) {
```

LinearSieve XorBasis SubsetSumSqrt

```
    for(int i = B-1; i>=0; i--) x = max(x, x ^ b[i]);
    return x;
}
// k-th smallest subset xor (1-indexed, 1st = 0)
long long kth(long long k) {
    if (k < 1 || k > (1LL << sz)) return -1;
    long long x = 0, cnt = 1LL << sz;
    for (int i = B-1; i >= 0; i--) if (b[i]) {
        if (k > cnt/2) {
            if (!(x >> i & 1)) x ^= b[i]; k -= cnt/2;
        } else {
            if (x >> i & 1) x ^= b[i];
        }
        cnt /= 2;
    }
    return x;
}
// Count how many subset xors are < x
long long count_lt(long long x) {
    if (x < 0) return 0;
    long long ans = 0, cnt = 1LL << sz;
    long long mask = 0;
    for (int i = B-1; i >= 0; i--) {
        if (b[i]) {
            if (x >> i & 1) {
                ans += cnt/2;
                if (!(mask >> i & 1)) mask ^= b[i];
            } else if (mask >> i & 1) {
                mask ^= b[i];
            }
            cnt /= 2;
        }
        else if ((x >> i & 1) != (mask >> i & 1)) {
            return (x >> i & 1) ? ans + cnt : ans;
        }
    }
    return ans;
}
};
```

SubsetSumSqrt.h

Description: ...
Time: ... f9ccce, 28 lines

```
main:
// Sum of elements <= N implies that every element is <= N
vector<int> freq(N + 1, 0);
for (int i = 0; i < N; i++) {
    int x; cin >> x; freq[x]++;
}
vector<pair<int, int>> compressed;
for (int i = 1; i <= N; i++) {
    if (freq[i] > 0) compressed.emplace_back(i, freq[i]);
}
vector<int> dp(N + 1, 0); dp[0] = 1;
for (const auto &[w, k] : compressed) {
    vector<int> ndp = dp;
    for (int p = 0; p < w; p++) {
        int sum = 0;
        for(int mult = p, cnt = 0; mult<=N; mult += w, cnt++){
            if (cnt > k) {
                sum -= dp[mult - w * cnt]; cnt--;
            }
            if (sum > 0) ndp[mult] = 1; sum += dp[mult];
        }
    }
    swap(dp, ndp);
}
cout << "Possible subset sums are:\n";
for (int i = 0; i <= N; i++) {
```

```
    if (dp[i] > 0) cout << i << " ";
}
```